



Cinema Reservation System

Microservices-based Application

Alessio Fior

Matricola: 0381477

**Università degli Studi di Roma "Tor
Vergata"**

Project Objectives

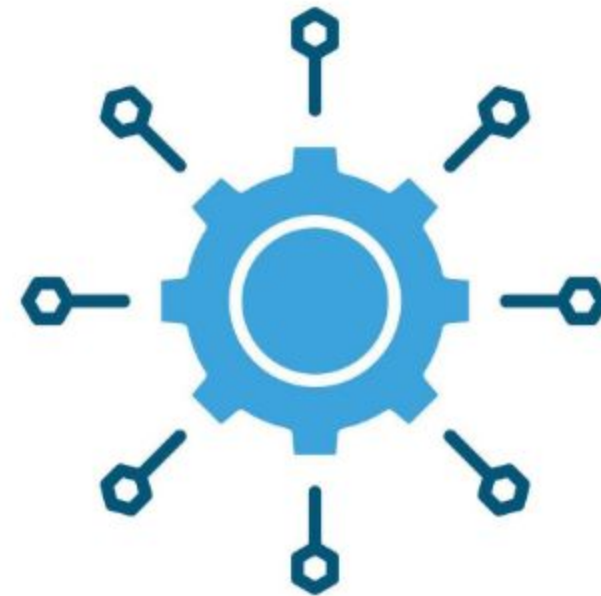
 **Architecture Evolution:** Transition from monolithic to microservices.

 **High Concurrency:** Managing simultaneous reservation requests.

 **Resilience:** Ensuring fault tolerance across components.

 **Consistency:** Maintaining availability and data integrity.

 **Deployment:** Containerization on AWS EC2 infrastructure.



System Architecture



API GATEWAY

Routing, gRPC translation, and circuit breaker protection.



CATALOG SERVICE

Manages movie metadata and projections via gRPC.



BOOKING SERVICE

Core logic: Redis locks, NATS JetStream, Event Sourcing.



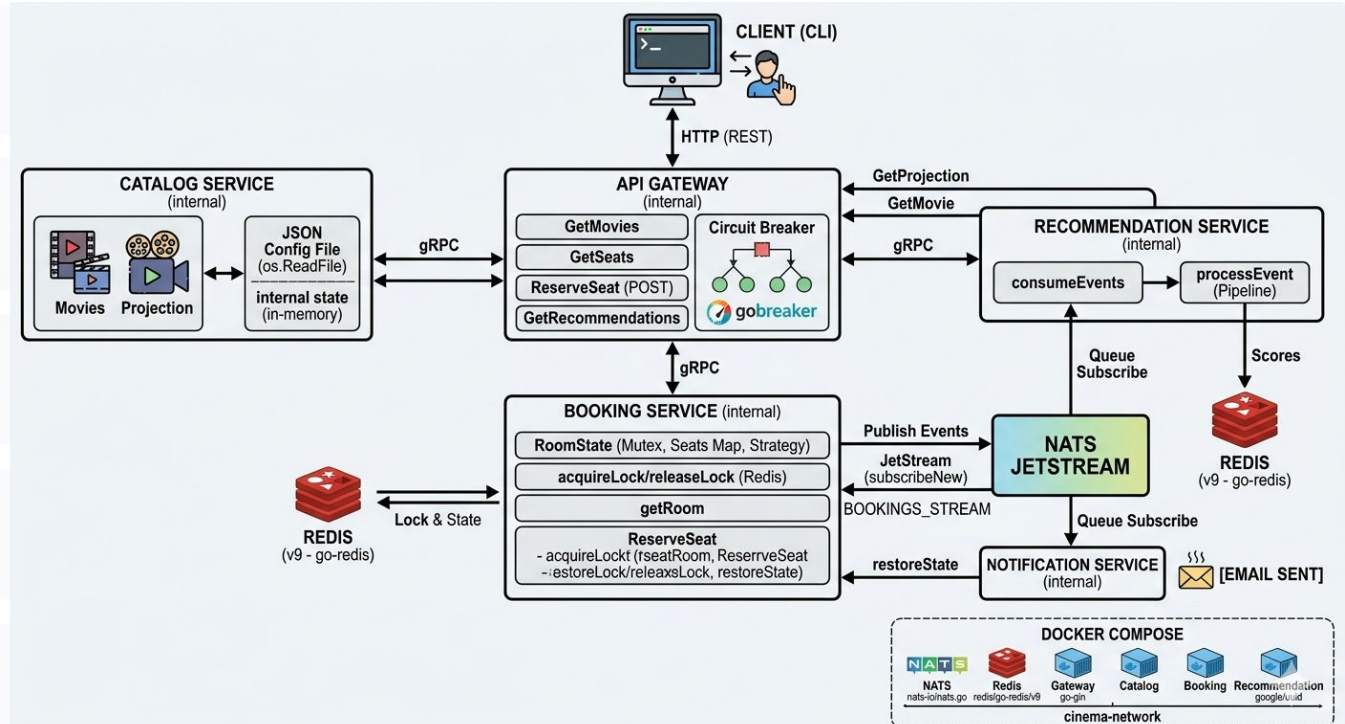
NOTIFICATION SERVICE

Asynchronous email delivery through NATS events.



RECOMMENDATION SERVICE

User rankings based on historical booking data.



Technological Stack



REDIS

Synchronization layer and distributed locks (SETNX) to prevent race conditions during booking.



NATS JETSTREAM

Event-driven backbone for asynchronous communication and event replay for fault tolerance.



gRPC

High-performance framework for internal synchronous calls between the Gateway and backend services.



DOCKER

Containerization to ensure consistency across environments and simplify deployment on AWS EC2.

Circuit Breaker Pattern

Problem: Prevent cascading failures if the *Booking Service* is slow or offline.

Tool: gobreaker library integrated into the **API Gateway**.

Closed

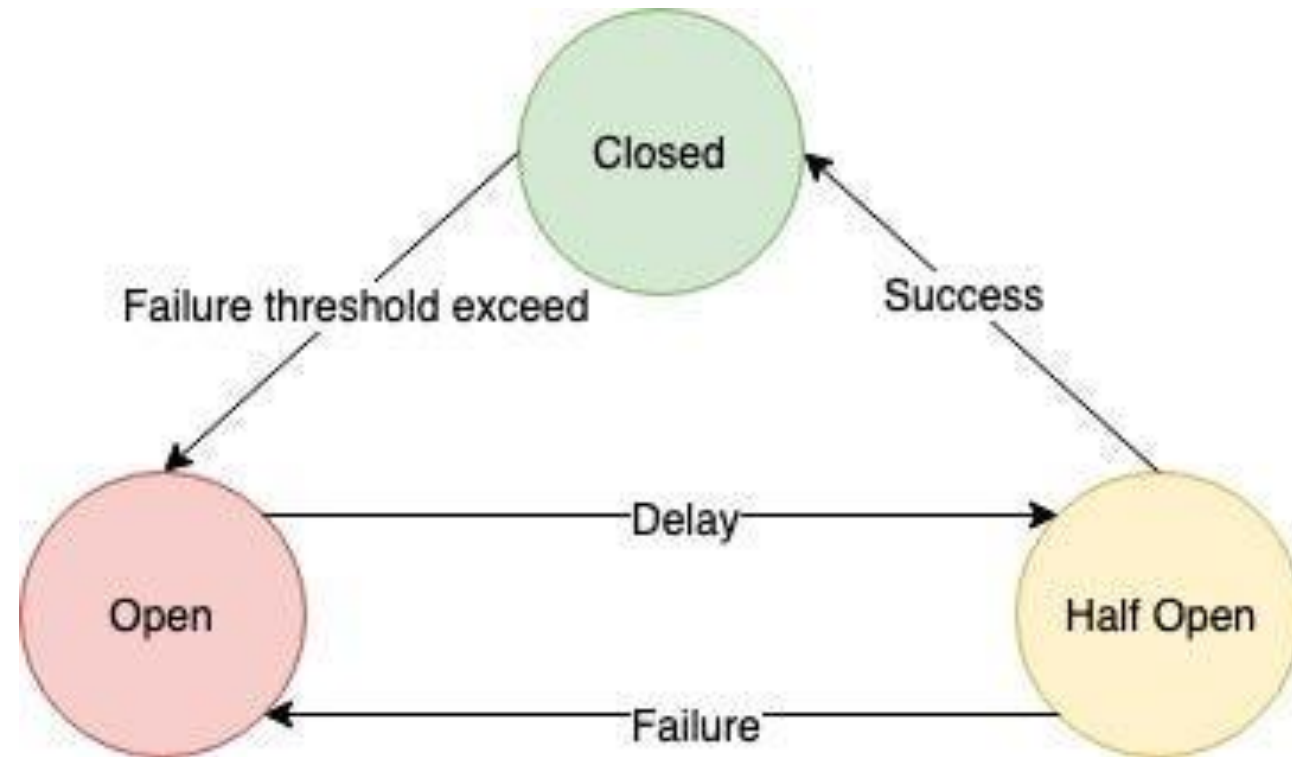
Normal operation. Requests are forwarded.

Open

Rejection after **3 failures**. Circuit opens for **15s**.

Half-Open

Recovery test phase. Limited requests allowed.



Booking & Concurrency

Distributed Level (Redis)

Distributed Locking via atomic SETNX operations to coordinate instances.

Local Level (Go)

Internal thread synchronization using Mutex primitives.

Redis Technical Details



Unique Lock Key

lock:projectionID:seatID



TTL (Time To Live)

Automatic expiration for safety.

Event-Driven Communication

Paradigm: Asynchronous communication based on **Pub/Sub** events.

Publisher: Booking Service

Subject: bookings.event.

Consumers: Notification, Recommendation

Scalability Mechanism: *NATS Queue Groups*

Key Advantages



Loose Coupling

Independent service evolution.



Horizontal Scalability

Scale instances as needed.



Fault Isolation

Decoupled failure management.

Event Sourcing Pattern

Key Concept: The system state (occupied seats) is derived from a sequence of **immutable events**, rather than being stored in a traditional DB.

🔗 Anatomy of an Event

```
{  
  "BookingID": "uuid",  
  "UserID": "uuid",  
  "ProjectionID": "int",  
  "SeatID": "int"  
}
```

🔄 Recovery Mechanism

- 1 NATS JetStream acts as a **Persistent Log**.
- 2 On startup, the service **replays** all history.
- 3 In-memory room state is **fully rebuilt**.

Strategy Design Pattern

GoF Definition: *"Defines a family of algorithms, encapsulates each one, and makes them interchangeable."*

```
type SeatStrategy interface {  
    Validate(room *Room, seat int32) error  
}
```

Default Strategy

Basic availability check.

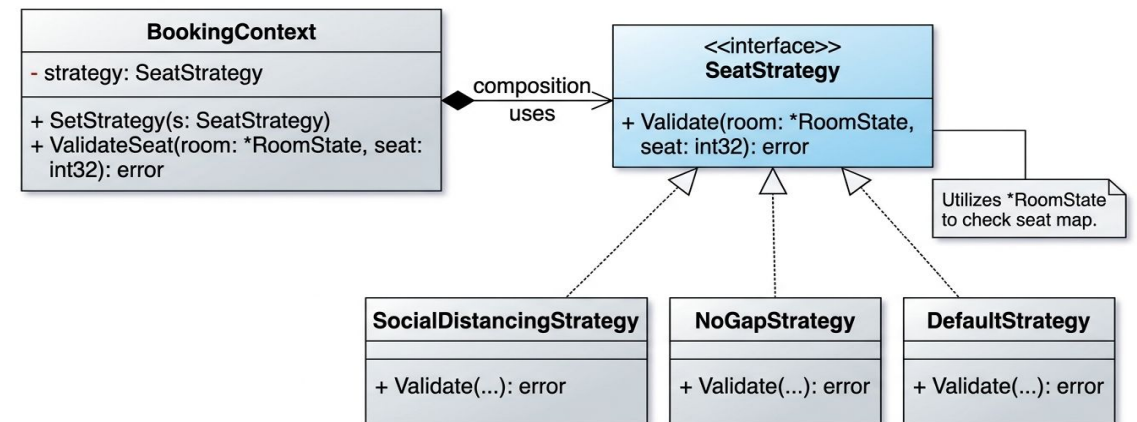
Social Distancing Strategy

One empty seat between users.

No Gap Strategy

Prevents isolated empty seats.

Package: internal



Test 1: Circuit Breaker Validation

```
for i in {1..5}; do
  curl -X POST http://localhost:8080/book \
  -H "Content-Type: application/json" \
  -d '{"projection_id":"p101","seat_id":1,"user_id":"u1"}'
done
```

```
booking-1 exited with code 2
gateway-1 | 2026/05/09 06:20:14 [CB] failures=1 requests=1
gateway-1 | [GIN] 2026/05/09 - 06:20:14 | 500 | 649.894µs | 172.
gateway-1 | 2026/05/09 06:20:14 [CB] failures=2 requests=2
gateway-1 | [GIN] 2026/05/09 - 06:20:14 | 500 | 270.606µs | 172.
gateway-1 | 2026/05/09 06:20:14 [CB] failures=3 requests=3
gateway-1 | 2026/05/09 06:20:14 [CB] closed -> open
gateway-1 | [GIN] 2026/05/09 - 06:20:14 | 500 | 228.963µs | 172.
gateway-1 | [GIN] 2026/05/09 - 06:20:14 | 503 | 133.049µs | 172.
```

Test 2: Concurrency Validation

```
for i in {1..20}; do
    curl -s -X POST http://localhost:8080/book \
        -H "Content-Type: application/json" \
        -d "{ \"projection_id\": \"p201\", \"seat_id\": 25,
        \"user_id\": \"user$i\" }" &
done
wait
```

[illegible]

Test 3: Fault Tolerance & Recovery

```
docker stop notification-service
# Perform seat reservation while service is offline
docker start notification-service
```

 **Durability:** Events stored in *NATS JetStream*.

 **Recovery:** Consumer replays missed events on restart.

 **Result:** Correct delivery of pending notifications.

```
notification-service-1 exited with code 0
booking-1          | 2026/05/09 08:15:07 Test
gateway-1          | [GIN] 2026/05/09 - 08:15:07 | 200 | 48.87ms | 172.21.0.1 | POST    "/book"
booking-1          | 2026/05/09 08:15:24 Test
gateway-1          | [GIN] 2026/05/09 - 08:15:24 | 200 | 3.08ms | 172.21.0.1 | POST    "/book"
notification-service-1 | 2026/05/09 08:15:37 [NOTIFICATION] Ready, listening for new events...
notification-service-1 | 2026/05/09 08:15:37 Notification Service started successfully.
notification-service-1 | 2026/05/09 08:15:37 [E] [EMAIL SENT] Confirmation for user1: Seat 5 successfully booked!
notification-service-1 | 2026/05/09 08:15:37 [E] [EMAIL SENT] Confirmation for user2: Seat 13 successfully booked!
```

Test 4: Replication & State Recovery

```
docker compose up --scale booking=3
```



Scalability: gRPC load balancing across replicas.



Resilience: Gateway redirects requests on failure.



Event Sourcing: Instances reconstruct state from JetStream.



Result: Consistent state post-crash/restart.

```
booking-1 | 2026/05/09 11:35:44 Test
booking-1 | 2026/05/09 11:35:44 ✓ [EVENT PUBLISHED] Subject: bookings.event.p101 | Data: {"booking_id":"RES-p101-1","user_id":"user1","projection_id":"p101","seat_id":1}
notification-service-1 | 2026/05/09 11:35:44 ✉ [EMAIL SENT] Confirmation for user1: Seat 1 successfully booked!
gateway-1 | [GIN] 2026/05/09 - 11:35:44 | 200 | 14.52ms | 172.21.0.1 | POST "/book"
booking-1 exited with code 2
booking-2 | 2026/05/09 11:36:28 Test
booking-2 | 2026/05/09 11:36:28 ✓ [EVENT PUBLISHED] Subject: bookings.event.p102 | Data: {"booking_id":"RES-p102-2","user_id":"user2","projection_id":"p102","seat_id":2}
gateway-1 | [GIN] 2026/05/09 - 11:36:28 | 200 | 4.78ms | 172.21.0.1 | POST "/book"
booking-3 exited with code 2
booking-2 exited with code 2
gateway-1 | [GIN] 2026/05/09 - 11:37:44 | 409 | 2.58ms | 172.21.0.1 | POST "/book"
booking-1 exited with code 2 (restarting)
booking-1 | 2026/05/09 11:38:23 Connected to Redis: redis:6379
booking-1 | 2026/05/09 11:38:23 [EVENT SOURCING] restoring state...
booking-1 | 2026/05/09 11:38:24 Booking Service running on :50051
booking-1 | 2026/05/09 11:38:24 NATS: nats://nats:4222
booking-1 | 2026/05/09 11:38:24 Redis: redis:6379
booking-1 | 2026/05/09 11:38:40 Test
gateway-1 | [GIN] 2026/05/09 - 11:38:40 | 409 | 2.69ms | 172.21.0.1 | POST "/book"
```

Limitations & Future Work

Limitations

-  **API Gateway Bottleneck**
Single entry point under extreme traffic (requires horizontal scaling).
-  **gRPC Coupling**
Synchronous blocking communication patterns between core services.
-  **In-Memory State**
Runtime state is kept in memory (Event Sourcing handles durability, but snapshotting is missing).

Future Improvements

-  **Identity Management**
Full Auth, persistent profiles, and security.
-  **Transactional Payments**
Integration of payment gateways.
-  **AI Recommendations**
ML-based adaptive user suggestions.
-  **Storage Migration**
Transitioning Catalog to SQL/NoSQL DBs.
-  **Observability**
Centralized logging and distributed tracing.